Although this works, it's not ideal. Some of the best places for interrupting a thread are where it's blocked waiting for something, which means that the thread isn't running in order to call `interruption_point()`! What you need here is a means for waiting for something in an interruptible fashion.

9.2.3. Interrupting a condition variable wait

OK, so you can detect interruptions at carefully chosen places in your code, with explicit calls to `interruption_point()`, but that doesn't help when you want to do a blocking wait, such as waiting for a condition variable to be notified. You need a new function—`interruptible_wait()`—which you can then overload for the various things you might want to wait for, and you can work out how to interrupt the waiting. I've already mentioned that one thing you might be waiting for is a condition variable, so let's start there: what do you need to do in order to be able to interrupt a wait on a condition variable? The simplest thing that would work is to notify the condition variable once you've set the interrupt flag, and put an interruption point immediately after the wait. But for this to work, you'd have to notify all threads waiting on the condition variable in order to ensure that your thread of interest wakes up. Waiters have to handle spurious wake-ups anyway, so other threads would handle this the same as a spurious wake-up—they wouldn't be able to tell the difference. The `interrupt_flag` structure would need to be able to store a pointer to a condition variable so that it can be notified in a call to `set()`. One possible implementation of `interruptible_wait()` for condition variables might look like the following listing.

Listing 9.10. A broken version of `interruptible_wait` for `std::condition_variable`